

HAKIZIMANA Emmanuel

Reg No: 224019436

UR-CBE

African Centre of Excellence in Data Science (ACE-DS)

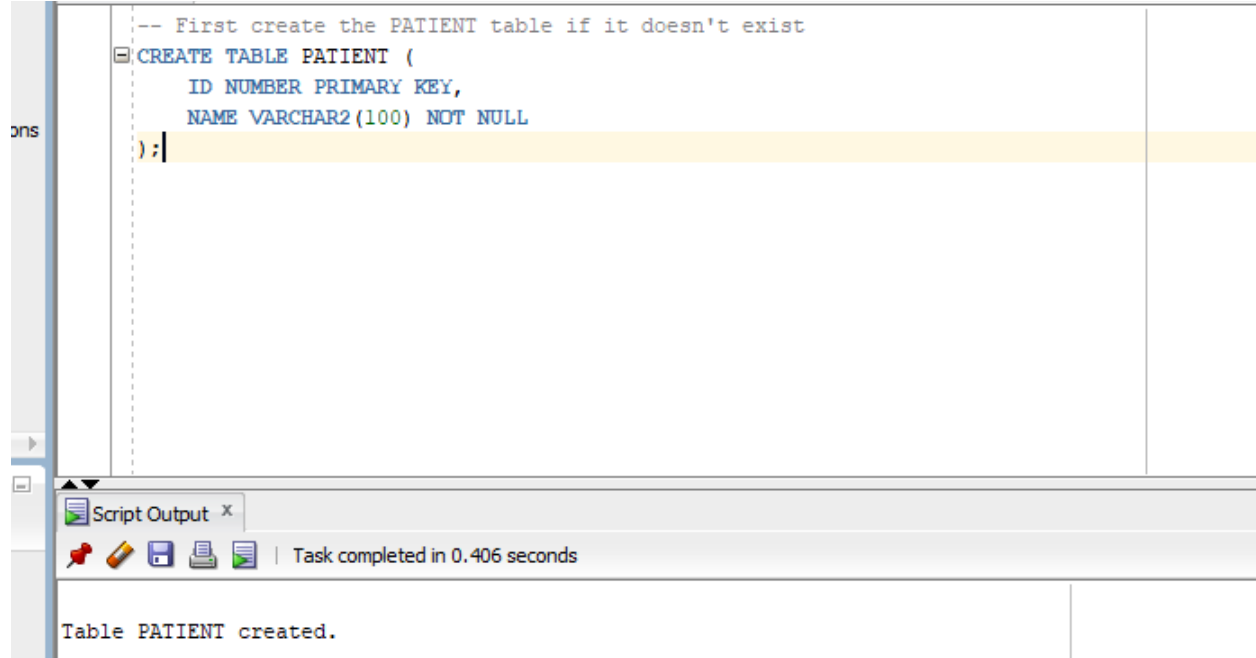
Masters of Data Science in Mining

Advanced Database and Technology

30th October, 2025

INTELLIGENT-DATABASE ASSIGNMENT SCREENSHOT

Screenshot 1: Patient table

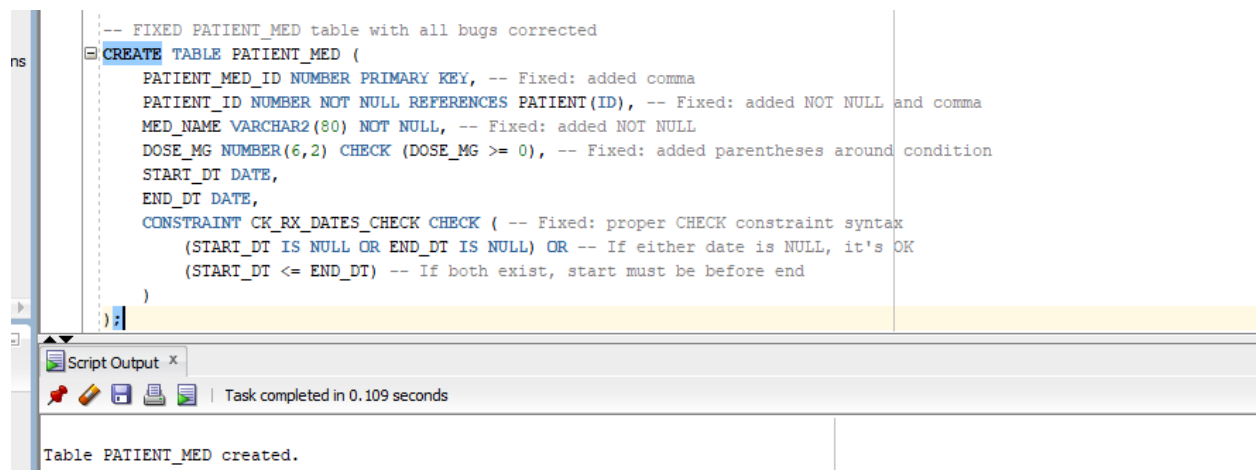


The screenshot shows a database IDE with a script editor and a script output window. The script editor contains the following SQL code:

```
-- First create the PATIENT table if it doesn't exist
CREATE TABLE PATIENT (
    ID NUMBER PRIMARY KEY,
    NAME VARCHAR2(100) NOT NULL
);
```

The script output window shows the message: "Table PATIENT created." and "Task completed in 0.406 seconds".

Screenshot2: Patient_MED

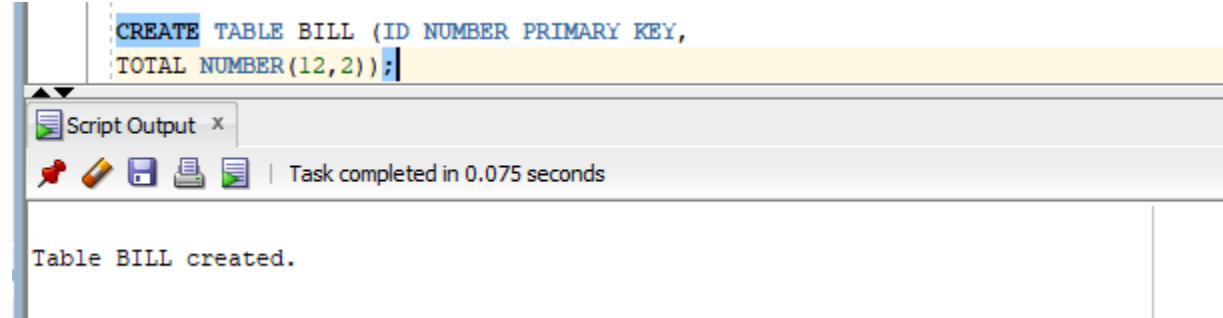


The screenshot shows a database IDE with a script editor and a script output window. The script editor contains the following SQL code:

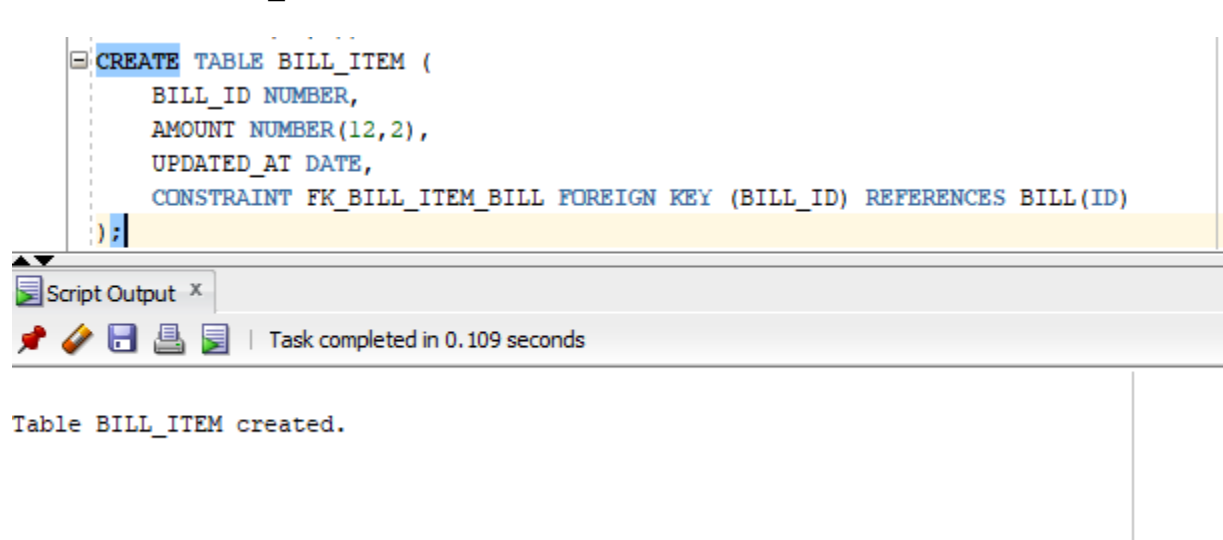
```
-- FIXED PATIENT_MED table with all bugs corrected
CREATE TABLE PATIENT_MED (
    PATIENT_MED_ID NUMBER PRIMARY KEY, -- Fixed: added comma
    PATIENT_ID NUMBER NOT NULL REFERENCES PATIENT(ID), -- Fixed: added NOT NULL and comma
    MED_NAME VARCHAR2(80) NOT NULL, -- Fixed: added NOT NULL
    DOSE_MG NUMBER(6,2) CHECK (DOSE_MG >= 0), -- Fixed: added parentheses around condition
    START_DT DATE,
    END_DT DATE,
    CONSTRAINT CK_RX_DATES_CHECK CHECK ( -- Fixed: proper CHECK constraint syntax
        (START_DT IS NULL OR END_DT IS NULL) OR -- If either date is NULL, it's OK
        (START_DT <= END_DT) -- If both exist, start must be before end
    )
);
```

The script output window shows the message: "Table PATIENT_MED created." and "Task completed in 0.109 seconds".

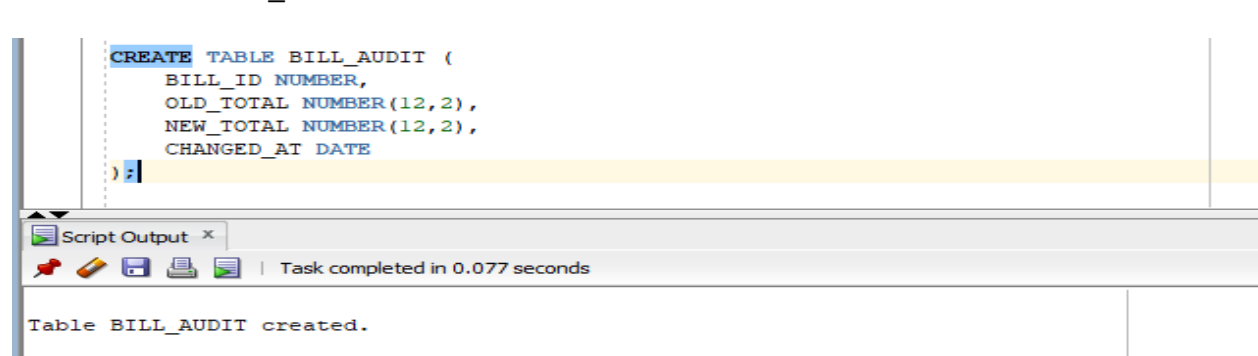
Screenshot 3: Bill Table



Screenshot 4: Bill_ITEM



Screenshot 5: Bill_Audit



Screenshot 6: Trigger compiled

```
CREATE OR REPLACE TRIGGER TRG_BILL_TOTAL_CMP
FOR INSERT OR UPDATE OR DELETE ON BILL_ITEM
COMPOUND TRIGGER

    -- Declaration section: store affected bill IDs
    TYPE BILL_ID_TABLE IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
    g_bill_ids BILL_ID_TABLE;

Script Output x
Task completed in 0.377 seconds

Trigger TRG_BILL_TOTAL_CMP compiled
```

Screenshot 7: TRG_BILL_TOTAL_CMP compiled

```
-- Before each statement: clear the collection
BEFORE STATEMENT IS
BEGIN
    g_bill_ids.DELETE;
END BEFORE STATEMENT;

-- After each row: collect affected bill IDs
AFTER EACH ROW IS
BEGIN
    -- For INSERT: use new bill ID

Script Output x
Task completed in 0.113 seconds

Trigger TRG_BILL_TOTAL_CMP compiled
```

Screenshot 8: Updated bill

```
ELSIF UPDATING THEN
    g_bill_ids(:NEW.BILL_ID) := 1;
    g_bill_ids(:OLD.BILL_ID) := 1;
    -- For DELETE: use old bill ID
ELSIF DELETING THEN
    g_bill_ids(:OLD.BILL_ID) := 1;
END IF;
END AFTER EACH ROW;

Script Output x
Task completed in 0.11 seconds

Trigger TRG_BILL_TOTAL_CMP compiled
Trigger TRG_BILL_TOTAL_CMP compiled
Trigger TRG_BILL_TOTAL_CMP compiled
Trigger TRG_BILL_TOTAL_CMP compiled
Trigger TRG_BILL_TOTAL_CMP compiled
Trigger TRG_BILL_TOTAL_CMP compiled
```

Screenshot 9: Testing completed

```
-- TEST: Mixed DML operations
-- Insert some items
INSERT INTO BILL_ITEM
VALUES
(1, 100.00, SMALL);
INSERT INTO BILL_ITEM
VALUES
(1, 50.00, SMALL);
INSERT INTO BILL_ITEM
VALUES
(2, 200.00, SMALL);
COMMIT;
```

Script Output x | Task completed in 0.06 seconds

Commit complete.

Screenshot 10: Staff_Supervisors

```
CREATE TABLE STAFF_SUPERVISORS (
  EMPLOYEE VARCHAR2(50),
  SUPERVISOR VARCHAR2(50)
);
```

Script Output x | Task completed in 0.061 seconds

Table STAFF_SUPERVISORS created.

Screenshot 11: Testing committed successful

```
-- Insert test data
INSERT INTO STAFF_SUPERVISOR VALUES ('Alice', 'Bob');
INSERT INTO STAFF_SUPERVISOR VALUES ('Bob', 'Carol');
INSERT INTO STAFF_SUPERVISOR VALUES ('Carol', 'David');
INSERT INTO STAFF_SUPERVISOR VALUES ('David', NULL); -- CEO has no supervisor
INSERT INTO STAFF_SUPERVISOR VALUES ('Eve', 'Bob');
INSERT INTO STAFF_SUPERVISOR VALUES ('Frank', 'Eve');
COMMIT;
```

Script Output x | Task completed in 0.071 seconds

Commit complete.

Screenshot 12: Fixed Recursive Query

```
WITH SUPERS (EMP, SUP, HOPS, PATH) AS (
  -- Anchor: Start with each employee and their immediate supervisor
  SELECT
    EMPLOYEE,
    SUPERVISOR,
    1 AS HOPS,
    TO_CHAR(EMPLOYEE) AS PATH
  FROM STAFF_SUPERVISOR
  WHERE SUPERVISOR IS NOT NULL

  UNION ALL
  -- Recursive: climb up the chain
  SELECT
    S.EMP,
    T.SUPERVISOR,
    S.HOPS + 1,
    S.PATH || '-'>' || T.SUPERVISOR
  FROM SUPERS S
  JOIN STAFF_SUPERVISOR T ON S.SUP = T.EMPLOYEE
  WHERE T.SUPERVISOR IS NOT NULL
  AND INSTR(S.PATH, T.SUPERVISOR) = 0
)
SELECT * FROM SUPERS
ORDER BY EMP, HOPS;
```

Query Result x

SQL | All Rows Fetched: 0 in 0.036 seconds

EMP	SUP	HOPS	PATH
-----	-----	------	------

Screenshot 13: Triple table created

```
CREATE TABLE TRIPLE (
  S VARCHAR2(100),
  P VARCHAR2(50),
  O VARCHAR2(100)
);
```

Script Output x

Task completed in 0.109 seconds

Table TRIPLE created.

Screenshot 14: Sample data inserted

```
-- Diagnosis facts
INSERT INTO TRIPLE VALUES ('Patient1', 'hasDiagnosis', 'Influenza');
INSERT INTO TRIPLE VALUES ('Patient2', 'hasDiagnosis', 'CommonCold');
INSERT INTO TRIPLE VALUES ('Patient3', 'hasDiagnosis', 'Diabetes'); -- Not infectious!
INSERT INTO TRIPLE VALUES ('Patient4', 'hasDiagnosis', 'COVID19');

-- Taxonomy (classification) facts
INSERT INTO TRIPLE VALUES ('Influenza', 'isA', 'ViralInfection');
INSERT INTO TRIPLE VALUES ('CommonCold', 'isA', 'ViralInfection');
INSERT INTO TRIPLE VALUES ('COVID19', 'isA', 'ViralInfection');
INSERT INTO TRIPLE VALUES ('ViralInfection', 'isA', 'InfectiousDisease');
COMMIT;
```

Script Output x

Task completed in 0.079 seconds

Commit complete.

Screenshot 15: Fixed Recursive query

```
WITH ISA(CHILD, ANCESTOR) AS (
  -- Anchor: direct "isA" relationships
  SELECT S AS CHILD, O AS ANCESTOR
  FROM TRIPLE
  WHERE P = 'isA'

  UNION ALL

  -- Recursive: climb up the taxonomy hierarchy
  SELECT I.CHILD, T.O AS ANCESTOR
  FROM ISA I
  JOIN TRIPLE T ON I.ANCESTOR = T.S
  WHERE T.P = 'isA'
)
SELECT DISTINCT
  T.S AS PATIENT_ID
FROM TRIPLE T
JOIN ISA ON T.O = ISA.CHILD
WHERE T.P = 'hasDiagnosis'
AND ISA.ANCESTOR = 'InfectiousDisease';
```

Query Result x

SQL | All Rows Fetched: 0 in 0.003 seconds

PATIENT_ID

Screenshot 16: Clinic Table created

```
CREATE TABLE CLINIC (
  ID NUMBER PRIMARY KEY,
  NAME VARCHAR2(100),
  GEOM SDO_GEOMETRY
);
```

Script Output x

Task completed in 0.295 seconds

Table CLINIC created.

Screenshot 17: Sample inserted in Clinic table

```
-- Insert some sample clinics around Kigali, Rwanda coordinates
INSERT INTO CLINIC VALUES (
  1, 'City Clinic',
  SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(30.0610, -1.9500, NULL), NULL, NULL)
);
INSERT INTO CLINIC VALUES (
  2, 'Central Hospital',
  SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(30.0580, -1.9550, NULL), NULL, NULL)
);
INSERT INTO CLINIC VALUES (
  3, 'North Medical',
  SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(30.0550, -1.9450, NULL), NULL, NULL)
);
INSERT INTO CLINIC VALUES (
  4, 'South Clinic',
  SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(30.0650, -1.9650, NULL), NULL, NULL)
);
COMMIT;
```

Script Output x

Task completed in 0.097 seconds

Commit complete.

Screenshot 18: Spatial index created

```
DELETE FROM USER_SDO_GEOM_METADATA WHERE TABLE_NAME = 'CLINIC';
INSERT INTO USER_SDO_GEOM_METADATA VALUES (
  'CLINIC', 'GEOM',
  SDO_DIM_ARRAY(
    SDO_DIM_ELEMENT('LONG', -180, 180, 0.5),
    SDO_DIM_ELEMENT('LAT', -90, 90, 0.5)
  ),
  4326
);
COMMIT;
```

Script Output x

Task completed in 0.079 seconds

Commit complete.

Screenshot 19: Index clinic

```
-- FIXED: Query 1 - Clinics within 1 km
SELECT C.ID, C.NAME
FROM CLINIC C
WHERE SDO_WITHIN_DISTANCE(
  C.GEOM,
  SDO_GEOMETRY(
    2001,
    4326, -- Fixed: Correct SRID for WGS84 (was 3857)
    SDO_POINT_TYPE(30.0600, -1.9570, NULL), -- Fixed: Correct order (longitude, latitude)
    NULL,
    NULL
  ),
  'distance=1 unit=KM' -- Fixed: Added unit specification
) = 'TRUE';
```

Query Result x

All Rows Fetched: 0 in 0.258 seconds

ID	NAME
----	------

Screenshot 20: Clinic with distance

```
-- FIXED: Query 2 - Nearest 3 clinics with distances
SELECT C.ID, C.NAME,
  ROUND(SDO_GEOM.SDO_DISTANCE(
    C.GEOM,
    SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(30.0600, -1.9570, NULL), NULL, NULL),
    0.005, -- Tolerance
    'unit=KM' -- Fixed: Specify units
  ), 2) AS DISTANCE_KM
FROM CLINIC C
ORDER BY DISTANCE_KM
FETCH FIRST 3 ROWS ONLY;
```

Query Result x

All Rows Fetched: 0 in 0.094 seconds

ID	NAME	DISTANC...
----	------	------------